

Chapitre 3 : Synchronisation et communication Inter-Processus

Les processus concurrents s'exécutant dans le système d'exploitation peuvent être des processus coopératifs ou indépendants.

- Un processus est indépendant s'il n'affecte pas les autres processus ou ne peut pas être affecté par eux. Un processus qui ne partage pas de données avec d'autres processus est indépendant
- Un processus est coopératif s'il peut affecter les autres processus en cours d'exécution ou être affecté par eux. Un processus qui partage des données avec d'autres processus est un processus coopératif. Les données partagées par les processus coopératifs peuvent se trouver en mémoire principale ou en mémoire secondaire dans un fichier. Les accès concurrents à des données partagées peuvent produire des incohérences de données.

Les processus ont alors besoin de communiquer entre eux pour se passer les informations et être sûr qu'ils n'interviennent pas les uns avec les autres au moment d'exécuter des activités critiques. La communication entre processus doit se faire de façon structurée, sans utiliser les interruptions.

Le SE doit fournir aux processus coopératifs les moyens de communiquer entre eux par l'intermédiaire d'une fonction de communication inter processus (IPC : InterProcess communication).

2.1 Conditions de concurrence

Situation où plusieurs processus accèdent et manipulent en concurrence les mêmes données et où le résultat de l'exécution dépend de l'ordre dans lequel on accède aux instructions.

Les processus concurrents doivent parfois partager données (fichiers ou mémoire commune) et ressources (On parle donc de tâches coopératives). Il faut donc interdire les processus de lire/écrire des données partagées au même moment. Pour cela, nous avons besoin de l'exclusion mutuelle (Mutex: mutual exclusion)

Si l'accès n'est pas contrôlé, le résultat de l'exécution du programme pourra dépendre de l'ordre d'entrelacement de l'exécution des instructions (non-déterminisme). Un programme pourra donner des résultats différents et parfois indésirables.

2.2 Section Critique

La section critique est le segment de code dans lequel le processus peut accéder aux données partagées et apporter des modifications.

Il n'y a pas d'entrelacement avec autres processus. Une fois qu'une tâche y entre, il faut lui permettre de terminer cette section sans permettre à autres tâches de jouer sur les mêmes données.

Quand un processus exécute sa section critique, aucun autre processus n'est autorisé à exécuter la sienne.

Lorsqu'un processus manipule une ressource partagée, nous disons qu'il se trouve dans une section critique (SC) associée à cette ressource.

Chaque processus doit donc demander une permission avant d'entrer dans une section critique (SC).

La structure du segment de code est le suivant:

- La section de code qui effectue cette requête est la **section d'entrée**

- La section critique est normalement suivie d'une **section de sortie**
- Le code qui reste est la **section restante** (SR): non-critique

```
repeat
    section d'entrée
    section critique
    section de sortie
    section restante
forever
```

Le problème de la section critique est de trouver un algorithme d'exclusion mutuelle de processus dans l'exécution de leur Section Critiques afin que le résultat de leurs actions ne dépendent pas de l'ordre d'entrelacement de leur exécution (avec un ou plusieurs processeurs)

- L'exécution des sections critiques doit être mutuellement exclusive: à tout instant, un seul processus peut exécuter une SC pour une variable donnée (même lorsqu'il y a plusieurs processeurs)
- Ceci peut être obtenu en plaçant des instructions spéciales dans les sections d'entrée et sortie.

2.3 Exclusion Mutuelle

Un Mutex (anglais : Mutual exclusion, Exclusion mutuelle) est une primitive de synchronisation utilisée en programmation informatique pour éviter que des ressources non partagées d'un système ne soient utilisées en même temps. Son implémentation varie selon les systèmes (masquage des interruptions, lecture/écriture en un cycle, etc.)

Elle permet d'assurer que si un processus utilise une variable ou fichier partagés, les autres processus seront exclus de la même activité. Ces algorithmes permettent de réguler l'accès aux données, par exemple pour qu'une routine ne s'exécute qu'une seule fois.

Certains algorithmes utilisent un état pour commander l'exécution : les mutex doivent savoir si les programmes concernés sont occupés (busy) ou s'ils ont terminé et sont en attente (wait). De tels algorithmes sont par exemple :

- l'algorithme de Dekker(1965) ;
- l'algorithme de Peterson(1981).

La plupart des mutex ont des effets secondaires, par exemple les sémaphores (mutex avec un compteur) qui peuvent bloquer l'exécution, créer des goulets d'étranglement, voire ne pas remplir leur rôle en permettant tout de même l'accès aux données protégées. Un autre effet est le blocage total des ressources, si le programme qui les utilisait n'a pas informé le système qu'il n'en avait plus besoin.

2.4 Verrous

Un mécanisme proposé pour permettre de résoudre l'exclusion mutuelle d'accès à une ressource est le mécanisme de verrou (en anglais lock). Un verrou est un objet système sur lequel deux opérations sont définies. Un tel objet peut s'assimiler à une ressource logicielle, les opérations permettant d'acquérir ou de libérer cette ressource.

- verrouiller (v) permet au processus d'acquérir le verrou v s'il est disponible. S'il n'est pas disponible, le processus est bloqué en attente de la ressource.
- déverrouiller (v) permet au processus de libérer le verrou v qu'il possédait. Si un ou plusieurs processus étaient en attente de ce verrou, un seul de ces processus est réactivé et reçoit le verrou.

En tant qu'opérations systèmes, ces opérations sont indivisibles, c'est-à-dire que le système garantit l'absence d'interférence entre leurs exécutions par plusieurs processus. On dit encore que ce sont des primitives.

À fin que des processus avec des variables partagées puissent réussir, il est nécessaire que tous les processus impliqués utilisent le même algorithme de coordination (Un protocole commun)

2.5 Sémaphores

Un sémaphore est un mécanisme proposé par E.W.Dijkstra en 1965, et qui est un peu plus général que le verrou. Il se présente comme un distributeur de jetons, mais le nombre de jetons est fixe et non renouvelable: les processus doivent restituer leur jeton après utilisation. S'il y a un seul jeton en circulation, on retrouve le verrou. Les deux opérations sur un sémaphore sont traditionnellement dénotées P(s) et V(s).

- P(s) ou down(s) permet à un processus d'obtenir un jeton, s'il y en a de disponibles. Si aucun n'est disponible, le processus est bloqué.
- V(s) ou up(s) permet à un processus de restituer un jeton. Si des processus étaient en attente de jeton, l'un d'entre eux est réactivé et le reçoit.

Un sémaphore S est un entier qui, sauf pour l'Initialisation, est accessible seulement par ces 2 opérations atomiques et mutuellement exclusives:

- wait(S)
- signal(S)
- Il est partagé entre tous les processus qui s'intéressent à la même section critique
- Les sémaphores seront présentés en deux étapes:
 - sémaphores qui sont occupés à attendre (busy waiting)
 - sémaphores qui utilisent des files d'attente
- On fait distinction aussi entre sémaphores compteurs et sémaphores binaires, mais ce derniers sont moins puissants.

2.6 Communication entre processus

Lorsque des processus ont besoin d'échanger des informations, ils peuvent le faire par l'intermédiaire d'une zone de mémoire commune.

Ex : les tubes Unix (en anglais pipes) qui sont des mécanismes de communication interprocessus.

2.7 Interblocage

La synchronisation et la communication des processus peuvent conduire à un autre problème. Supposons qu'un premier processus P1 demande et obtienne la ressource imprimante, seule ressource de ce type. Un second processus P2 demande ensuite et obtient la ressource dérouleur de bande, également seule de ce type. Si P1 demande ultérieurement la ressource dérouleur de bande sans avoir libéré la ressource imprimante, il est bloqué en attendant la libération par P2. Si maintenant, P2 demande la ressource imprimante avant de libérer la ressource dérouleur de bande, il est bloqué en attendant la libération par P1 de cette ressource. On voit que chacun des processus est bloqué, en attendant que l'autre libère la ressource qu'il possède. On dit qu'il y a un interblocage entre les processus (en anglais deadlock).

On peut donner une représentation graphique à l'interblocage. Représentons les ressources par des rectangles, et les processus par des cercles. On met une flèche d'une ressource vers un processus lorsque la ressource est allouée au processus, et une flèche d'un processus vers une ressource lorsque le processus attend cette ressource. La figure 3.1 représente l'évolution du graphe de notre exemple précédent conduisant à l'interblocage.

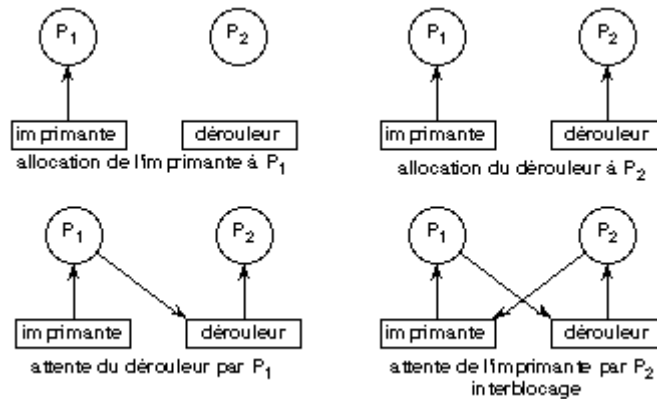
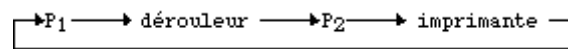


Fig. 3.1 Allocation conduisant à l'interblocage.

On peut constater que dans le graphe final, il y a un cycle représenté par:



En fait, tout interblocage est signalé par la présence d'un tel cycle dans ce graphe, et inversement, ce qui explique l'intérêt de cette représentation.

Le problème de l'interblocage est résolu de diverses façons par les systèmes.

- La première méthode est la politique de l'autruche: elle consiste à ignorer le problème. Considérant que le coût des autres méthodes est assez élevé, il peut s'avérer rentable de ne pas résoudre le problème lorsque la probabilité d'avoir un interblocage est faible. S'il survient, il faut relancer le système.
- La deuxième méthode est une méthode de détection-guérison. Elle consiste à vérifier de temps en temps la présence de tels cycles, et à tuer un ou plusieurs processus jusqu'à la disparition de ces cycles. Chaque destruction d'un tel processus entraîne évidemment la restitution des ressources qu'il possédait.
- La troisième méthode est une méthode de prévention. Elle consiste à imposer des contraintes sur les demandes des processus de telle sorte qu'un tel cycle ne puisse jamais se produire. Ainsi, si les deux processus ci-dessus demandaient leurs ressources dans le même ordre (par exemple imprimante puis dérouteur), le deuxième processus serait bloqué sur la demande de la première ressource, sans n'avoir encore obtenu aucune ressource, et il ne pourrait y avoir de cycle.
- La quatrième méthode est une méthode d'évitement. Elle consiste à prévoir à l'avance les différents cas où un cycle pourrait se former, et à éviter l'interblocage en retardant une allocation qui serait possible, jusqu'à être certain qu'il n'y a pas de risque d'interblocage. La figure 3.2 montre sur notre exemple qu'en retardant l'allocation du dérouteur à P_2 jusqu'à ce que P_1 ait libéré l'imprimante, on permet à celui-ci d'acquérir le dérouteur avant P_2 et on évite ainsi la création du cycle. Il faut évidemment savoir à l'avance quels seront les besoins des processus dans ce cas.

Un processus qui est en attente d'une ressource peut rester bloqué indéfiniment sans être pour autant en situation d'interblocage. On dit que l'on a une situation de famine (en anglais starvation), lorsqu'un ou plusieurs processus n'obtiennent pas les ressources dont ils ont besoin par suite du comportement de l'ensemble des processus, sans être en situation d'interblocage. La différence essentielle réside dans le fait qu'il est possible de sortir de la situation de famine par suite du changement du comportement des processus, alors qu'il n'est pas possible de sortir d'un interblocage sans retirer autoritairement une ressource à un des processus en interblocage.